

Component Differences with PWA Kit

PWA Kit components are typically JavaScript + PropTypes with Chakra primitives and style props. Storefront Next components are TypeScript-first and generally composed from Radix/shadcn patterns with utility-class styling and CVA-based variants. The main shift is from runtime styling and theme objects to compile-time class composition and typed component contracts.

Contents

Component Architecture

[Styling Comparison](#)[Responsive Design Patterns](#)[Component Variants](#)[Key Similarities](#)[Conversion Considerations](#)[Component Library Conversion](#)[TypeScript Adoption](#)

Component Architecture

Aspect	PWA Kit	Storefront Next
Language	JavaScript + PropTypes	TypeScript
Type Checking	Runtime (PropTypes)	Compile-time (TypeScript)
Component Library	Chakra UI (runtime)	shadcn/ui (copied source)
Styling Approach	CSS-in-JS (Emotion)	Utility classes (Tailwind)
Variant System	Theme config	Class Variance Authority
Primitives	Chakra components	Radix UI headless

Styling Comparison

Aspect	PWA Kit (Chakra)	Storefront Next (Tailwind)
Inline styles	<code><Box padding="4" bg="blue.500"></code>	<code><div className="bg-blue-500"></code>
Responsive	<code>display={{base: 'none', lg: 'block'}}</code>	<code>className="lg:block"</code>

[Try for free](#)

states	blue-60	
Theme tokens	color="gray.500"	className="gray-500"
Component styles	useMultiStyleConfig('Button')	buttonVariant
Style override	Props spread: {...styles.container}	Class method: cn(baseClassName)

Responsive Design Patterns

PWA Kit:

```
1 // Object syntax for breakpoints
2 <Box display={{base: 'block', md: 'flex', lg: 'grid'}}
3
4 // Array syntax (base, sm, md, lg, xl)
5 <Grid templateColumns={['1fr', '1fr', 'repeat(2, 1fr)']}
6
7 // Hook-based
8 const [isDesktop] = useMediaQuery('(min-width: 992px)')
```

Storefront Next:

```
1 // Breakpoint prefixes (mobile-first)
2 <div className="block md:flex lg:grid">
3
4 // Responsive grid
5 <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3">
6
7 // JavaScript detection
8 const isDesktop = typeof window !== 'undefined' &&
9   window.matchMedia('(min-width: 1024px)').matches
```

Component Variants

PWA Kit (Theme-based):

```
1 // In theme/components/button.js
2 export default {
3   baseStyle: {...},
4   variants: {
5     solid: {...},
6     outline: {...},
7   },
8   sizes: {
9     sm: {...},
10  }
```



```
15 <Button variant=solid size=md>Click</Button>
```

Storefront Next (CVA):

```
1 // In component file
2 const buttonVariants = cva("base-classes", {
3   variants: {
4     variant: { solid: "...", outline: "..." },
5     size: { sm: "...", md: "..." },
6   },
7 });
8
9 // Usage
10 <Button variant="solid" size="md">
11   Click
12 </Button>;
```

Key Similarities

- Composition over inheritance: Both use functional components composed from primitives
- Props passthrough: Both spread `...rest` props to underlying elements
- Responsive-first: Both prioritize mobile-first responsive design
- Variant systems: Both support component variants for consistent styling
- Accessibility: Both build on accessible component foundations (Chakra/Radix)
- HOC patterns: Both use HOCs for cross-cutting concerns (auth, suspense)

Conversion Considerations

Here are some tips for moving from Chakra to Tailwind.

- Style props vs. Utility classes: Convert `padding="4"` to `className="p-4"`.
- Theme tokens vs. Tailwind config: Map color/spacing scales.
- `useMultiStyleConfig` vs. CVA: Convert component style objects to variant functions.
- Responsive objects vs. breakpoint prefixes: Replace `{{base: 'x', lg: 'y'}}` with `"x lg:y"`.

Component Library Conversion

[Try for free](#)

Box	<code><div></code> with Tailwind	Useful for creating a container
Flex	<code><div className="flex"></code>	Access flexbox
HStack	<code><div className="flex flex-row gap-*"></code>	Horizontal
VStack	<code><div className="flex flex-col gap-*"></code>	Vertical
Stack	<code><div className="flex gap-*"></code>	Useful for flex
StackDivider	<code><Separator></code> (ui/separator)	Optional class
Center	<code><div className="flex items-center justify-center"></code>	
Container	<code><div className="container mx-auto"></code>	
Grid	<code><div className="grid"></code>	Access grid
GridItem	<code><div></code>	Useful for grid
SimpleGrid	<code><div className="grid grid-cols-*"></code>	
Wrap	<code><div className="flex flex-wrap"></code>	
WrapItem	<code><div></code>	Divide wrap
Spacer	<code><div className="flex-1"></code>	Optional padding
AspectRatio	<code><AspectRatio></code> (ui/aspect-ratio)	Responsive
Divider	<code><Separator></code> (ui/separator)	Responsive

[Try for free](#)

Text	<code><p></code> , <code></code> with Tailwind	Useful for
Heading	<code><h1></code> - <code><h6></code> with Tailwind	Useful for
Buttons & Links		
Button	<code><Button></code> (ui/button)	Component for buttons
ButtonGroup	<code><div className="flex gap-2"></code>	Group of buttons
IconButton	<code><Button size="icon"></code>	Useful for icons
Link	React Router <code><Link></code>	Outgoing links
CloseButton	<code><Button size="icon" variant="ghost"></code>	Window close button
Form Controls		
Input	<code><Input></code> (ui/input)	
InputGroup	<code><div className="relative"></code>	Parent container
InputLeftElement	<code><div className="absolute left-3"></code>	Prefix element
InputRightElement	<code><div className="absolute right-3"></code>	Suffix element
Select	<code><SelectNative></code> (ui/select-native)	Native select element
Checkbox	<code><Checkbox></code> (ui/checkbox)	Radio button
Radio	<code><RadioGroup></code> (ui/radio-group)	Radio button
RadioGroup	<code><RadioGroup></code> (ui/radio-group)	Radio button
FormControl	<code><div></code>	Useful for form

[Try for free](#)

FormErrorMessage	<code><p className="text-destructive text-sm"></code>	
useNumberInput	Custom implementation	Not needed
Feedback		
Alert	<code><Alert></code> (ui/alert)	
AlertIcon	Icon inside <code><Alert></code>	Used for icons
AlertTitle	<code><AlertTitle></code> (ui/alert)	
AlertDescription	<code><AlertDescription></code> (ui/alert)	
Spinner	<code><div className="animate-spin"></code>	Optional for loading states
Skeleton	<code><Skeleton></code> (ui/skeleton)	
useToast	Custom toast implementation	Optional for toast notifications
Overlay		
Modal	<code><Dialog></code> (ui/dialog)	Required for modal dialogs
ModalOverlay	Built into <code><Dialog></code>	Automatic overlay
ModalContent	<code><DialogContent></code> (ui/dialog)	
ModalHeader	<code><DialogHeader></code> (ui/dialog)	
ModalBody	<code><div></code> inside DialogContent	
ModalFooter	<code><DialogFooter></code> (ui/dialog)	
ModalCloseButton	Built into <code><DialogContent></code>	Optional for close button
AlertDialog	<code><AlertDialog></code> (ui/alert-dialog)	Required for alert dialogs
AlertDialogOverlay	Built into <code><AlertDialog></code>	
AlertDialogContent	<code><AlertDialogContent></code>	



AlertDialogFooter	<AlertDialogFooter>	
AlertDialogBody	<AlertDialogDescription>	
Drawer	<Drawer> (ui/drawer)	Vue 3
DrawerOverlay	Built into <Drawer>	
DrawerContent	<DrawerContent>	
DrawerHeader	<DrawerHeader>	
DrawerBody	<div> inside DrawerContent	
DrawerFooter	<DrawerFooter>	
DrawerCloseButton	<DrawerClose>	
Popover	<Popover> (ui/popover)	React
PopoverTrigger	<PopoverTrigger>	
PopoverContent	<PopoverContent>	
PopoverHeader	<div> with styling	Next.js
PopoverBody	<div> inside PopoverContent	
PopoverFooter	<div> with styling	Next.js
PopoverArrow	CSS-based	Tailwind CSS
PopoverCloseButton	<Button> with X icon	
Tooltip	<Tooltip> (ui/tooltip)	React
Portal	React Portal or Radix Portal	Bulma
Disclosure		
Accordion	<Accordion> (ui/accordion)	React
AccordionItem	<AccordionItem>	
AccordionButton	<AccordionTrigger>	

[Try for free](#)

AccordionIcon	Built into <AccordionTrigger>	Clipboard icon
useDisclosure	React state or Radix open state	Control state using useDisclosure hook
Navigation		
Breadcrumb	<Breadcrumb> (ui/breadcrumb)	
BreadcrumbItem	<BreadcrumbItem>	
BreadcrumbLink	<BreadcrumbLink>	
Menu (dropdown)	<DropdownMenu> (ui/dropdown-menu)	Radix
Navigation Menu	<NavigationMenu> (ui/navigation-menu)	Radix
Data Display		
Badge	<Badge> (ui/badge)	Clipboard icon
Image / Img	 with Tailwind	Use className="w-10 h-10" for square images
Avatar	<Avatar> (ui/avatar)	Radix
List	or	
ListItem		
Card	<Card> (ui/card)	Clipboard icon
Other		
Icon	Lucide React icons	Clipboard icon, Icon, '1'
VisuallyHidden		Screen reader



		cc o
Hooks		
useMultiStyleConfig	CVA + cn()	Di cc
useStyleConfig	CVA + cn()	
useTheme	Tailwind CSS variables	Ac cc
useMediaQuery	window.matchMedia()	O br
useBreakpoint	Tailwind breakpoints	Re pr
useBreakpointValue	Tailwind breakpoints	U sy
useRadio / useRadioGroup	Radix RadioGroup	Bl cc
useStyles	Not needed	St br
createStylesContext	React Context	O pr
StylesProvider	React Context	O di
extendTheme	tailwind.config.js	Ex th
ChakraProvider	Not needed	Ta tir

TypeScript Adoption

- Convert PropTypes to TypeScript interfaces.
- Add explicit return types to functions.
- Use `React.ComponentProps<'element'>` for HTML element props.
- Replace `PropTypes.shape({...})` with `interface` definitions.



Try for free



DEVELOPER CENTERS

Heroku
MuleSoft
Tableau
Commerce Cloud
Lightning Design System
Einstein
Quip

POPULAR RESOURCES

Documentation
Component Library
APIs
Trailhead
Sample Apps
AppExchange

COMMUNITY

Trailblazer Community
Events and Calendar
Partner Community
Blog
Salesforce Admins
Salesforce Architects

© Copyright 2026 Salesforce, Inc. [All rights reserved](#). Various trademarks held by their respective owners. Salesforce, Inc. Salesforce Tower, 415 Mission Street, 3rd Floor, San Francisco, CA 94105, United States

[Legal](#) [Terms of Service](#) [Privacy Information](#) [Responsible Disclosure](#) [Trust](#) [Contact](#) [Use of Cookies](#)

[Cookie Preferences](#)  [Your Privacy Choices](#)

